PROJECT GUIDE · PART 3

# StreetCare AI Run & Test Guide

Set it up once, build it in four commands, run all **50 test cases**, then try **every endpoint** of the live service with the right input and with each wrong one, and follow one report from the queue to a works order, end to end.

Setup in 4 steps | 4 build commands | 50 test cases | 11 endpoints | 76 captured examples | End-to-end walk-through | Mac · Linux · Windows

---

Every command and reply in this guide was captured from a real run of the project built from the Day 1 commit plus the twenty Day 2–5 zips · Wrenford Borough Council · Street Care Office

# The whole guide in 5 steps

**1** **Set up** venv · install · .env → **2** **Build** 4 commands, once → **3** **Test** pytest tests.py → **4** **Serve** API + dashboard → **5** **Try it** every endpoint

## How to read the boxes

```
$ python run_api_service.py
```

Dark box = a command. Type everything after the `$` (or `>` on Windows) and press Enter. Grey lines starting with # are notes.

**✓ YOU SHOULD SEE**

```
50 passed in 23.86s
```

Green-edged box = what the terminal printed in the captured run. Timings and random IDs (REV-…, RUN-…, tokens) will differ on yours.

**Windows** = PowerShell **Mac / Linux** = Terminal **All** = same everywhere

**GOLDEN RULE**

Run every command from **inside the `Project` folder** (the one with `tests.py` and the six `run_*.py` files), with the virtual environment **active**. Every path in `config.py` is relative to that folder.

## What you need first

| Item | Details |
|---|---|
| Python 3.11 – 3.14 | Check with `python --version`. pandas 3 needs 3.11 or newer. The captured run used 3.14. |
| Internet (first time) | To install packages and to download the embedding model `all-MiniLM-L6-v2` (~90 MB) once, during knowledge indexing. |
| A Gemini API key + model name | Free from Google AI Studio. Needed only by the review agents and the chat; the build, 40 of the 50 tests and every non-AI endpoint run without it. |
| ~2 GB free disk | PyTorch (used by the embedding model) is the biggest package. |
| Ports 8081 and 8080 free | 8081 = REST service ( `config.API_PORT` ), 8080 = dashboard ( `.streamlit/config.toml` ). |

## What you get at the end

**50 / 50** tests pass with a Gemini key (40 without one)

**40** open reports ranked in the queue

**16** staff accounts across 3 roles

# Step 1 · Set up (only once)

## a Get the code and open a terminal in Project

Pull `main` after all twenty Day 2–5 branches are merged (or unzip each zip into the repository root: every zip already starts with `Project/`). Then open a terminal in the `Project` folder.

**Windows** In File Explorer click the address bar, type `powershell`, Enter (or VS Code: Terminal → New Terminal). **Mac / Linux** Right-click the folder → Open in Terminal.

## b Create and activate a virtual environment

**Windows · PowerShell**

```
> python -m venv venv
> venv\Scripts\Activate.ps1
```

**Mac / Linux**

```
$ python3 -m venv venv
$ source venv/bin/activate
```

**POWERSHELL SAYS 'RUNNING SCRIPTS IS DISABLED'?**

Run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (answer **Y**), then activate again.

## c Install the packages

```
$ pip install -r requirements.txt
$ pip install sentence-transformers
```

**DON'T SKIP THE SECOND LINE**

`requirements.txt` does not list `sentence-transformers`, and ChromaDB needs it to embed the passages. Without it `run_knowledge_indexing.py` stops with "*The sentence_transformers python package is not installed*". Better still, add the line `sentence-transformers` to `requirements.txt` in a small PR.

## d Put your Gemini key in Project/.env

The committed `.env` holds placeholders (`your api key` / `model name.`). Replace both values: no quotes, no spaces around `=`. The model below is the one the `.env` comment suggests; any current Gemini model name works.

```
$ GEMINI_API_KEY=paste_your_key_here
$ GEMINI_MODEL=gemini-3.1-flash-lite
```

**NEVER COMMIT THE REAL KEY**

`.env` is in `.gitignore`. If git still tracks it, run `git rm --cached Project/.env` once so your key stays on your machine.

**EVERY NEW TERMINAL LATER**

Just `cd` into Project and activate the venv again (step b, second line). No reinstall.

# Step 2 · Build everything (4 commands, in this order)

Each command builds one stage that the next one reads. Run them once; run a stage again only after changing its code. None of them needs the Gemini key.

| # | Command | Built on | Makes | Time |
|---|---|---|---|---|
| 1 | `python run_database_setup.py` | Day 1 | `database/streetcare.db` : 6 tables, 16 accounts | <1 s |
| 2 | `python run_data_pipeline.py` | Day 1 | `processed/*_clean.csv` (5 files) + `output/data_quality_report.json` | 4 s |
| 3 | `python run_model_training.py` | Days 2–3 | 3 models + 2 schemas in `output/models/` , `output/model_metrics.json` | 10 s |
| 4 | `python run_knowledge_indexing.py` | Day 2 | `database/chroma_store/` : 205 passages from 16 PDFs | 14 s* |

*First run also downloads the embedding model.

```
$ python run_database_setup.py
```

**✓ YOU SHOULD SEE**

```
database: database/streetcare.db
  tables: agent_runs, chat_turns, llm_calls, report_reviews, review_decisions, user_accounts
  account created  amelia   Amelia Hart      Highways Inspector
  account created  owen     Owen Parry       Highways Inspector
  account created  priya    Priya Shah       Highways Inspector
  account created  callum   Callum Reid      Highways Inspector
  account created  jonah    Jonah Ellis      Highways Inspector
  account created  freya    Freya Morgan     Highways Inspector
  account created  bilal    Bilal Hussain    Highways Inspector
  account created  nina     Nina Brooks      Highways Inspector
  account created  grace    Grace Turner     Street Care Coordinator
  account created  samuel   Samuel Okafor    Street Care Coordinator
  account created  harriet  Harriet Lane     Street Care Coordinator
  account created  declan   Declan Burke     Street Care Coordinator
  account created  maya     Maya Patel       Customer Services Adviser
  account created  ethan    Ethan Wright     Customer Services Adviser
  account created  zara     Zara Ahmed       Customer Services Adviser
  account created  lewis    Lewis Grant      Customer Services Adviser
  accounts created: 16, already present: 0
```

```
$ python run_data_pipeline.py
```

**✓ YOU SHOULD SEE · ROW COUNTS ARE EXACT**

```
report_history_train            32738 received ->  30569 kept  (2169 removed)
report_history_eval              8202 received ->   7648 kept  (554 removed)
repair_logs_train               26095 received ->  25537 kept  (558 removed)
repair_logs_eval                 6476 received ->   6337 kept  (139 removed)
statement_sentences             36661 received ->  35717 kept  (944 removed)
quality report written to output/data_quality_report.json
```

```
$ python run_model_training.py
```

**✓ YOU SHOULD SEE · EVERY MODEL ABOVE ITS MINIMUM (0.80 AUC · 0.86 R2 · 0.95 F1)**

```
training three models
  repeat          held-out ROC AUC 0.8351  (train 0.8329, gap -0.0022)
  crew hours      held-out R2 0.9025  MAE 0.4158 hours  (train 0.9181, gap +0.0156)
  sentence        held-out macro F1 0.9775  (train 0.9804, gap +0.0029)
metrics written to output/model_metrics.json
every model reaches the minimum the configuration sets for it
```

```
$ python run_knowledge_indexing.py
```

**✓ YOU SHOULD SEE (FIRST LINES)**

```
16 documents, 136 sections, 205 passages indexed in database/chroma_store
what an inspection records under each defect standard, read back from the index
  Carriageway  Defect-Standard-Carriageway.pdf
      When a report is sent for inspection on site, the inspector who visits records what this standard
needs to decide the repair: the depth and the width of the defect, measured with a straight edge and a tape,
a photograph taken from the footway, the location fixed to a house number, a lamp column or a junction, and
whether the defect lies in a utility's reinstatement.
  Footway  Defect-Standard-Footway.pdf
  … one passage per defect standard
```

**IF A BUILD STEP REFUSES**

- **Training** prints *No model was saved: … is below …* and saves nothing when any model misses its minimum: fix the cleaning or features, then run it again.
- **Indexing** refuses to write into a folder that holds something other than a vector store; delete `database/chroma_store` and run it again.
- **Database setup** can be run again safely: existing accounts are kept (*already present: 16*).

# Step 3 · Run the 50 test cases

```
$ python -m pytest tests.py -q
```

The suite reads what the four build steps wrote, starts the REST service inside the test process (no server needs to be running) and, for 10 tests, opens **one real review run** and asks **one real chat question**, so those make live Gemini calls (7 for the review, 1 for the chat).

**✓ YOU SHOULD SEE · WITH THE GEMINI KEY**

```
..................................................        [100%]
50 passed in 23.86s
```

**✓ WITHOUT A KEY · THESE 10 FAIL, AND THAT IS EXPECTED**

```
FAILED tests.py::test_agents_called_once_in_order_with_tokens - AssertionErro...
FAILED tests.py::test_review_keeps_tool_findings_and_open_recommendation - As...
FAILED tests.py::test_order_writes_works_order_and_resident_update - Assertio...
FAILED tests.py::test_workflow_paths_and_resume_from_checkpoint - AssertionEr...
FAILED tests.py::test_run_waits_for_appointed_inspector - AssertionError: The...
FAILED tests.py::test_decision_once_refusals_before_calls - AssertionError: T...
FAILED tests.py::test_chat_answers_from_documents_and_cites_them - AssertionE...
FAILED tests.py::test_chat_memory_holds_own_last_exchanges - AssertionError: ...
FAILED tests.py::test_adviser_hears_status_and_resident_update - AssertionErr...
FAILED tests.py::test_team_hears_review_record_and_decision - AssertionError:...
10 failed, 40 passed in 28.90s
```

Each of those 10 fails with the same root cause, the 502 the service returns when Gemini rejects the key: *"Opening a review of SC0052102 returned 502 with {"detail":"400 INVALID_ARGUMENT … API key not valid"}"*. Put the key in `.env` and they pass.

## What the 50 tests cover

| Area | Tests (prefix `test_`) | # | Gemini? |
|---|---|---|---|
| **Day 1 · cleaning** | cleaning_keeps_latest_revision_once, readings_made_standard_units, references_types_and_places_canonical, answers_codes_and_register_read_canonically, impossible_readings_blanked_bad_keys_refused, quality_report_accounts_for_rows, statement_text_cleaning_leaves_plain_words | 7 | no |
| **Day 2 · documents & screening** | every_document_gives_up_text, documents_cut_into_sections_and_passages, index_holds_every_passage_once, tools_read_whole_own_sections, chat_search_skips_casebook_nearest_first, each_defect_type_read_from_own_standard, requirements_keep_sentence_and_figures, twin_depth_figures_read_from_own_documents, band_and_thresholds_read_however_laid_out, unknown_type_refused_reading_repeatable, requirements_met_at_boundary, follow_on_questions_ask_only_answers_needed, linked_report_screened_under_both_standards, coordination_triggers_follow_register_and_rules, assessment_decisions_and_worked_example | 15 | no |

| | | | |
|---|---|---|---|
| **Day 3 · features & models** | schemas_leave_out_outcome_and_identifiers, counts_and_triggers_use_own_standards, inputs_follow_schema_order_with_gaps, models_fit_schema_and_fill_gaps, repeat_model_auc_and_calibration, crew_hours_model_r2_and_floor, repeat_rises_with_failures_and_conflicts, expected_hours_rise_on_busy_street_and_conflict, bands_and_overrun_follow_thresholds, sentence_model_macro_f1, sentence_predictions_reach_every_type, statements_split_into_ordered_sentences, sentences_link_requirements_and_readings | 13 | no |
| **Day 4 · agents & chat** | decorated_reply_read_whole, street_figures_need_five_reports | 2 | no |
| **Day 4 · workflow run** | agents_called_once_in_order_with_tokens, review_keeps_tool_findings_and_open_recommendation, order_writes_works_order_and_resident_update, workflow_paths_and_resume_from_checkpoint, run_waits_for_appointed_inspector, decision_once_refusals_before_calls, adviser_hears_status_and_resident_update, team_hears_review_record_and_decision | 8 | **yes** |
| **Day 4 · chat with Gemini** | chat_answers_from_documents_and_cites_them, chat_memory_holds_own_last_exchanges | 2 | **yes** |
| **Day 5 · service & access** | sign_in_and_out_decide_access, roles_reach_permissions_and_districts, queue_order_and_rows_match_detail | 3 | no |
| **Total** | | **50** | **10** |

## Useful test commands

| Command | Use it when |
|---|---|
| `python -m pytest tests.py -q` | Normal run of all 50 |
| `python -m pytest tests.py -q -x` | Stop at the first failure |
| `python -m pytest tests.py -q --lf` | Run only the tests that failed last time |
| `python -m pytest tests.py -q -k queue` | Run tests whose name contains a word (here: `queue_order_and_rows_match_detail`) |
| `python -m pytest tests.py -q -k "not chat and not agents"` | Skip by name |
| `python -m pytest tests.py -q -p no:warnings` | Hide library warnings |
| `python -m pytest tests.py -v` | One line per test with its name |

**START FROM AN EMPTY DATABASE WHEN THE REVIEW TESTS RUN OUT OF REPORTS**

Every full run reviews and orders one more **ORDER REPAIR** report, and a revision is reviewed only once. After about a dozen runs the suite says "*No report in the queue whose assessment is ORDER REPAIR is left unreviewed … Start again from an empty database.*" Reset it (this also clears paused runs):

```
# Mac / Linux
$ rm database/streetcare.db database/checkpoint.db
$ python run_database_setup.py
```

```
> Remove-Item database\streetcare.db, database\checkpoint.db
> python run_database_setup.py
```

# Step 4 · Start the service and the dashboard

## The REST service (terminal 1)

```
$ python run_api_service.py
```

**✓ YOU SHOULD SEE (ABOUT 15 SECONDS: THE QUEUE IS WORKED OUT ONCE AT START-UP)**

```
Interactive documentation: http://localhost:8081/docs
INFO:     Started server process [31548]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8081 (Press CTRL+C to quit)
INFO:     127.0.0.1:54946 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:54948 - "POST /auth/signin HTTP/1.1" 200 OK
INFO:     127.0.0.1:54950 - "GET /queue?defect_type=Street%20lighting HTTP/1.1" 200 OK
INFO:     127.0.0.1:54952 - "GET /reviews?status_value=REPAIR_ORDERED&status_value=REFERRED_FOR_COORDINATION
HTTP/1.1" 200 OK
INFO:     127.0.0.1:54954 - "GET /queue/SC0000000 HTTP/1.1" 404 Not Found
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [31548]
```

**✓ IF A BUILD STEP WAS SKIPPED, IT REFUSES TO START AND SAYS WHAT IS MISSING (EXIT CODE 1)**

```
The StreetCare AI service cannot start. Missing:
  processed/report_history_train_clean.csv  (produced by run_data_pipeline.py)
  processed/report_history_eval_clean.csv  (produced by run_data_pipeline.py)
  output/models/repeat_model.pkl  (produced by run_model_training.py)
  output/models/crew_hours_model.pkl  (produced by run_model_training.py)
  output/models/sentence_model.pkl  (produced by run_model_training.py)
  output/models/repeat_features.json  (produced by run_model_training.py)
  output/models/crew_hours_features.json  (produced by run_model_training.py)
  data/input/open_defect_reports.csv  (produced by the supplied data folder)
  data/input/report_documents.csv  (produced by the supplied data folder)
  data/input/street_works_register.csv  (produced by the supplied data folder)
  data/docs  (produced by the supplied data folder)
  database/chroma_store/chroma.sqlite3  (produced by run_knowledge_indexing.py)
  database/streetcare.db  (produced by run_database_setup.py)
exit=1
```

## The dashboard (terminal 2, service still running)

```
$ streamlit run run_dashboard.py
```

Open **http://localhost:8080**. If the service is not running, the dashboard says so, names `http://localhost:8081` and how to start it, and draws nothing else.

## The 16 seeded accounts

| Role | Usernames | Password | District | Dashboard pages |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| **Highways Inspector** | amelia, owen | `11111111` | Town Centre | Report queue · Report detail · Decision queue · Audit trail · Street care chat |
| | priya, callum | | North | |
| | jonah, freya | | South | |
| | bilal, nina | | Villages | |
| **Street Care Coordinator** | grace, samuel, harriet, declan | `22222222` | none | Report queue · Report detail · Audit trail · Street care chat |
| **Customer Services Adviser** | maya, ethan, zara, lewis | `33333333` | none | Street care chat |

## Try the dashboard in 6 clicks

1. Sign in as **grace**. **Report queue**: counts on top, footway reports first; click a row.
2. **Report detail** opens on it: Screening, Statement evidence, Report and Review tabs. Type what the review should look at (5–1000 characters) and start it; the spinner says 5 model calls are being made.
3. Sign out, sign in as an inspector of that report's district who did **not** open it (e.g. **owen** for Town Centre). **Decision queue** lists only reviews on his streets; pick a decision and give a reason of at least 10 characters.
4. The page reports the new status, e.g. "*SC0052109 is now inspection requested: an inspector visits the defect …*"
5. **Audit trail**: the decided review, its documents, its call budget (calls of expected, tokens, latency) and everyone's chat turns.
6. Sign in as **maya**: only **Street care chat** is offered. Ask about the report.

> **WHAT THE DASHBOARD CHECKS ITSELF**
>
> Only the service's own limits, so a form the service would refuse is never sent: request 5–1000 characters, reason 10–1000, question 3–1000 (longer cannot be typed), an inspector's own district, and no decision form on your own review. Everything else, every figure and every refusal, comes from the service. A 401 anywhere ends the session and shows "*Your session ended: this account has signed in somewhere else or signed out …*".

# Step 5 · Test the live service

Leave `python run_api_service.py` running. Every example from here on is a real request and the reply the service gave. Pick one of three ways to send them.

### Option A · the browser (easiest)

1. Open **http://localhost:8081/docs**.
2. **POST /auth/signin** → Try it out → `{"username":"grace","password":"22222222"}` → Execute → copy `session_token`.
3. On any other endpoint, paste the token into the **X-Session-Token** header field, fill the parameters, Execute.

### Option B · Mac / Linux / Git Bash (curl) — used for every example in this guide

Run this once in a second terminal. It signs in the five people the examples use and keeps their tokens in variables ( `$TG` grace, `$TA` amelia, `$TO` owen, `$TP` priya, `$TM` maya).

```
$ B=http://localhost:8081
$ signin() { curl -s -X POST $B/auth/signin -H "Content-Type: application/json" -d "
{\"username\":\"$1\",\"password\":\"$2\"}" | python3 -c "import sys,json;print(json.load(sys.stdin)
['session_token'])"; }
$ TG=$(signin grace 22222222); TA=$(signin amelia 11111111); TO=$(signin owen 11111111)
$ TP=$(signin priya 11111111); TM=$(signin maya 33333333); TS=$(signin samuel 22222222)
# keep a review id from a reply:
$ REV=$(curl -s -X POST "$B/reviews" -H "X-Session-Token: $TA" -H "Content-Type: application/json" -d
'{"report_reference":"SC0052125","coordinator_request":"Are the requirements met?"}' | python3 -c
"import sys,json;print(json.load(sys.stdin)['review_id'])")
```

> **SIGNING IN AGAIN REPLACES YOUR TOKEN**
>
> An account holds one session. If you sign in as grace in the dashboard after setting `$TG`, `$TG` stops working (401 *Sign in first*). Run the `signin` line again.

### Option C · Windows PowerShell

```
> $B = "http://localhost:8081"
> function SignIn($u, $p) { (Invoke-RestMethod -Method Post "$B/auth/signin" -ContentType
"application/json" -Body (@{username=$u; password=$p} | ConvertTo-Json)).session_token }
> $TG = SignIn "grace" "22222222"; $TA = SignIn "amelia" "11111111"; $TO = SignIn "owen" "11111111";
$TM = SignIn "maya" "33333333"
> Invoke-RestMethod "$B/queue?defect_type=Footway" -Headers @{"X-Session-Token" = $TG}
> $body = @{report_reference="SC0052125"; coordinator_request="Are the requirements met?"} |
ConvertTo-Json
> $rev = Invoke-RestMethod -Method Post "$B/reviews" -Headers @{"X-Session-Token" = $TA} -ContentType
"application/json" -Body $body
> $d = @{decision="ORDER REPAIR"; decision_reason="Every requirement is met on the report."} |
ConvertTo-Json
> Invoke-RestMethod -Method Post "$B/reviews/$($rev.review_id)/decision" -Headers @{"X-Session-Token"
= $TO} -ContentType "application/json" -Body $d
```

**SEEING THE MESSAGE OF A REFUSED CALL IN POWERSHELL**

PowerShell shows 4xx/5xx replies as red errors. Wrap the call: `try { Invoke-RestMethod "$B/queue/SC0000000" -Headers @{"X-Session-Token"=$TG} } catch { $_.ErrorDetails.Message }`

# The 11 endpoints at a glance

| Method | Path | Permission | Who may call it | Success | Model calls |
|---|---|---|---|---|---|
| **POST** | `/auth/signin` | public | everyone | 200 | 0 |
| **POST** | `/auth/signout` | public (token in header) | everyone | 200 | 0 |
| **GET** | `/queue` | queue | Inspector, Coordinator | 200 | 0 |
| **GET** | `/queue/{report_reference}` | queue | Inspector, Coordinator | 200 | 0 |
| **POST** | `/reviews` | review | Inspector, Coordinator | 201 | 5 |
| **GET** | `/reviews` | audit | Inspector, Coordinator | 200 | 0 |
| **POST** | `/reviews/{review_id}/decision` | decision | Inspector only | 200 | 0 – 2 |
| **GET** | `/reviews/{review_id}/calls` | audit | Inspector, Coordinator | 200 | 0 |
| **POST** | `/chat/ask` | chat | all three roles | 200 | 1 |
| **GET** | `/chat/turns` | chat | all three roles | 200 | 0 |
| **GET** | `/audit/chat-turns` | audit | Inspector, Coordinator | 200 | 0 |

Every guarded endpoint needs the header `X-Session-Token`. A decision costs 2 more calls for ORDER REPAIR (works order + resident update), 1 for INSPECT ON SITE (inspection request), 0 for REFER FOR COORDINATION.

## Every status code the service answers, and why

| Code | Meaning | When it happens · body |
|---|---|---|
| 200<br>201 | Done (201 only for POST /reviews) | The JSON shape of the endpoint. |
| 401 | Who are you? | Sign-in failed (*Username or password is not correct*, same words for an unknown user and a wrong password) or no / unknown / replaced token (*Sign in first*). |
| 403 | Not allowed | The role lacks the route's permission (*The … role does not include the … permission*), checked before anything else, even for things that don't exist; or an inspector acting outside their district. |
| 404 | Not found | A report not in the queue, or an unknown review id. |
| 409 | Conflicts with the record | Revision already reviewed · review already decided · own review · decision not open · a different decision still being recorded. |
| 422 | Request has the wrong shape | FastAPI's own body: `detail` is a list naming each bad field (missing, too short, not one of the allowed values). Nothing is looked up, no call is spent. |

| | | |
|---|---|---|
| 502 | Something behind the service failed | A Gemini error, an agent reply that breaks its rules, a missing checkpoint, or a review outside the queue. `detail` is the failure's own message; a review that failed is marked **FAILED** and nothing is stored, so it can be started again. |

**ABOUT THE AI WORDING IN THE EXAMPLES**

The captured project had no Gemini key, so the 502 examples are the real replies to that. To show the success replies of the AI endpoints, the agents were answered by a **stand-in model**; those replies are marked **stand-in model wording**. With your key the agents' sentences differ, but every field, ID format, figure, status, call count and refusal is exactly as shown: they all come from the tools, the records and the rules, never from the model.

# POST /auth/signin · sign in

Public. Body: `username`, `password` (both required). Returns a fresh 32-character token; signing in again replaces the old one.

### 200 Sign in with a seeded account

```
$ curl -s -X POST "$B/auth/signin" -H "Content-Type: application/json" -d '{"username": "grace", "password": "22222222"}'
```


```
{
  "username": "grace",
  "display_name": "Grace Turner",
  "role": "Street Care Coordinator",
  "district": null,
  "session_token": "zj9sxnvAujJ44nsMYOIDRHjNIIBcqANq",
  "pages": ["Report queue", "Report detail", "… 2 more"],
  "permissions": ["queue", "review", "… 2 more"]
}
```


**How it is handled:** the password is hashed with the account's own salt (PBKDF2-SHA256, 200,000 rounds) and compared in constant time; `pages` and `permissions` come from the role. The password is never returned.

### 401 Wrong password

```
$ curl -s -X POST "$B/auth/signin" -H "Content-Type: application/json" -d '{"username": "grace", "password": "wrong-password"}'
```


```
{
  "detail": "Username or password is not correct"
}
```


**How it is handled:** same 401 and same words as an unknown user, so nobody can find out which usernames exist.

### 401 Unknown username

```
$ curl -s -X POST "$B/auth/signin" -H "Content-Type: application/json" -d '{"username": "nobody", "password": "22222222"}'
```


```
{
  "detail": "Username or password is not correct"
}
```


**How it is handled:** identical reply to the wrong password above.

### 422 Password left out

```
$ curl -s -X POST "$B/auth/signin" -H "Content-Type: application/json" -d '{"username": "grace"}'
```


```
{
  "detail": [{"type": "missing", "loc": ["body", "password"], "msg": "Field required", "input": {"username": "grace"}}]
}
```


**How it is handled:** refused by the request schema before the database is touched.

401 **Old token after the same account signed in again**

```
$ curl -s "$B/queue?defect_type=All" -H "X-Session-Token: $OLD_TS"
```

```
{
  "detail": "Sign in first"
}
```

**How it is handled:** samuel signed in a second time; the token he held before ( `$OLD_TS` ) no longer opens anything. One account = one session.

# POST /auth/signout · sign out

Public route, but the token travels in `X-Session-Token`.

200 **Sign out**

```
$ curl -s -X POST "$B/auth/signout" -H "X-Session-Token: $TS"
```

```
{
  "username": "samuel",
  "signed_out": true
}
```

**How it is handled:** the account's token is cleared.

401 **Sign out again with the same token**

```
$ curl -s -X POST "$B/auth/signout" -H "X-Session-Token: $TS"
```

```
{
  "detail": "Sign in first"
}
```

**How it is handled:** the token just ended opens nothing, not even another sign-out.

401 **Sign out with no token**

```
$ curl -s -X POST "$B/auth/signout"
```

```
{
  "detail": "Sign in first"
}
```

**How it is handled:** no header at all.

# GET /queue · the ranked queue

Query: `defect_type`, required, one of `All`, `Carriageway`, `Footway`, `Drainage`, `Vegetation`, `Street lighting` (URL-encode the space: `Street%20lighting`). Reports whose defect type *or linked type* is Footway come first; each group runs by reference. The queue is worked out once at start-up; only `report_status` is read live.

### 200 Whole queue

```
$ curl -s "$B/queue?defect_type=All" -H "X-Session-Token: $TG"
```


```
[
  {"queue_position": 1, "report_reference": "SC0052109", "revision_number": 1, "defect_type": "Footway",
"linked_defect_type": "No linked defect", "street_name": "Southfield Road", "district": "South",
"report_channel": "Contact centre", "report_status": "AWAITING_REVIEW", "document_count": 1,
"requirements_met_count": 3, "requirements_not_met_count": 1, "requirements_not_documented_count": 0,
"live_permit_count": 1, "conflicting_permit_count": 0, "coordination_trigger_count": 0, "assessment":
"INSPECT ON SITE", "repeat_likelihood": 0.04, "likelihood_band": "LOW", "expected_crew_hours": 2.1,
"overrun_expected": false, "disagreement_count": 0}
]
```


**How it is handled:** 40 rows, positions counting from 1. Shown: the first row of 40.

### 200 Footway only (defect type or linked type)

```
$ curl -s "$B/queue?defect_type=Footway" -H "X-Session-Token: $TG"
```


```
[
  {"queue_position": 1, "report_reference": "SC0052109", "revision_number": 1, "defect_type": "Footway",
"linked_defect_type": "No linked defect", "street_name": "Southfield Road", "district": "South",
"report_channel": "Contact centre", "report_status": "AWAITING_REVIEW", "document_count": 1,
"requirements_met_count": 3, "requirements_not_met_count": 1, "requirements_not_documented_count": 0,
"live_permit_count": 1, "conflicting_permit_count": 0, "coordination_trigger_count": 0, "assessment":
"INSPECT ON SITE", "repeat_likelihood": 0.04, "likelihood_band": "LOW", "expected_crew_hours": 2.1,
"overrun_expected": false, "disagreement_count": 0},
  "… 9 more"
]
```


**How it is handled:** same rows, same order and same positions as in the full queue, filtered by defect type or linked type.

### 422 A defect type with no standard

```
$ curl -s "$B/queue?defect_type=Graffiti" -H "X-Session-Token: $TG"
```


```
{
  "detail": [{"type": "literal_error", "loc": ["query", "defect_type"], "msg": "Input should be 'All',
'Carriageway', 'Footway', 'Drainage', 'Vegetation' or 'Street lighting'", "input": "Graffiti", "ctx":
{"expected": "'All', 'Carriageway', 'Footway', 'Drainage', 'Vegetation' or 'Street lighting'"}}]
}
```


**How it is handled:** not one of the six allowed values.

### 422 No defect_type at all

```
$ curl -s "$B/queue" -H "X-Session-Token: $TG"
```


```
{
  "detail": [{"type": "missing", "loc": ["query", "defect_type"], "msg": "Field required", "input": null}]
}
```


**How it is handled:** the filter is required; there is no default.

### 401 No token

```
$ curl -s "$B/queue?defect_type=All"
```

```
{
  "detail": "Sign in first"
}
```

**How it is handled:** the guard runs before the endpoint.

### 401 A made-up token

```
$ curl -s "$B/queue?defect_type=All" -H "X-Session-Token: not-a-real-token"
```

```
{
  "detail": "Sign in first"
}
```

**How it is handled:** a token no account holds is the same as none.

### 403 Customer Services Adviser

```
$ curl -s "$B/queue?defect_type=All" -H "X-Session-Token: $TM"
```

```
{
  "detail": "The Customer Services Adviser role does not include the queue permission"
}
```

**How it is handled:** advisers have only the **chat** permission; refused before anything is read.

# GET /queue/{report_reference} · everything about one report

Everything the review would work from, before any model call: the report as words, its documents, screening under its own standards, the street works register, the statement evidence and flags, and both model figures.

### 200 Detail of SC0052125

```
$ curl -s "$B/queue/SC0052125" -H "X-Session-Token: $TG"
```


```
{
  "queue_position": 3,
  "report_status": "AWAITING_REVIEW",
  "report": {"report_reference": "SC0052125", "report_channel": "StreetCare app", "revision_number": "1",
"reporter_type": "Resident", "street_name": "Market Place", "district": "Town Centre", "street_category":
"Traffic sensitive", "location_description": "opposite no. 50", "location_fixed": "Yes", "defect_type":
"Footway", "linked_defect_type": "No linked defect", "photograph_attached": "Yes", "defect_depth_mm": "20",
"defect_width_mm": "400", "water_depth_mm": "not given", "water_location": "not given", "water_reach_m": "not
given", "overhang_location": "not given", "clear_height_m": "not given", "on_highway_land": "not given",
"lighting_fault": "not given", "column_number_given": "not given", "column_lean_degrees": "not given",
"column_height_m": "not given", "previous_reports_at_location": "0", "planned_crew_hours": "2.5",
"supporting_document_count": "2", "live_street_works_on_street": "1", "conflicting_street_works": "0"},
  "report_documents": [{"document_id": "DOC-841233", "report_reference": "SC0052125", "report_channel":
"StreetCare app", "document_type": "Resident description", "document_title": "Resident description - Footway
- Market Place", "document_text": "Someone could easily trip on it, especially older residents. Just to say,
many people park along Market Place near the station. Please let me know what you deci…"}, "… 1 more"],
  "screening": {"report_reference": "SC0052125", "defect_types": ["Footway"], "requirements":
[{"defect_type": "Footway", "standard_document": "Defect-Standard-Footway.pdf", "requirement_kind":
"location_fixed", "requirement_name": "Location fixed", "requirement_sentence": "The location of the defect
is fixed to a house number, a lamp column number or a road junction.", "rules": {}, "result": "Met",
"blank_answers": []}, "… 3 more"], "requirements_met_count": 4, "requirements_not_met_count": 0,
"requirements_not_documented_count": 0, "coordination_triggers": [], "assessment": "ORDER REPAIR",
"decisions_open": ["ORDER REPAIR", "… 2 more"]},
  "register_count": {"live_street_works_on_street": 1, "conflicting_street_works": 0, "live_permits":
["SWP0071384"], "conflicting_permits": []},
  "evidence_findings": [{"document_id": "DOC-841233", "document_type": "Resident description", "sentence":
"Someone could easily trip on it, especially older residents.", "sentence_type": "Hazard",
"requirements_borne_on": []}, "… 8 more"],
  "disagreements": [],
  "likelihood": {"repeat_likelihood": 0.03, "likelihood_band": "LOW"},
  "crew_hours": {"expected_crew_hours": 3.1, "planned_crew_hours": 2.5, "overrun_expected": false}
}
```


**How it is handled:** screening, permits, evidence and figures come from the same tools the workflow uses, so the detail, the queue row and the stored review always agree. Lists trimmed to 1.

### 404 A report not in the queue

```
$ curl -s "$B/queue/SC0000000" -H "X-Session-Token: $TG"
```


```
{
  "detail": "No open defect report has the reference SC0000000"
}
```


**How it is handled:** reference not in the open queue.

### 403 Adviser asks for a detail

```
$ curl -s "$B/queue/SC0052125" -H "X-Session-Token: $TM"
```

```
{
  "detail": "The Customer Services Adviser role does not include the queue permission"
}
```

**How it is handled:** permission checked first, even for a report that exists.

# POST /reviews · run the five reviewing agents

Body: `report_reference`, `coordinator_request` (5–1000 characters after trimming). Runs the workflow to the gate: load context → planner → requirements analyst → evidence analyst → risk analyst → writer (5 model calls), stores the review as AWAITING_DECISION and pauses on disk. Answers **201**.

### 201 amelia opens the review of SC0052125

stand-in model wording

```
$ curl -s -X POST "$B/reviews" -H "X-Session-Token: $TA" -H "Content-Type: application/json" -d
'{"report_reference": "SC0052125", "coordinator_request": "Are the requirements of the standard met, and
does any document disagree with the report?"}'
```


```
{
  "review_id": "REV-ABDB38840557",
  "run_id": "RUN-2BC6397CA974",
  "thread_id": "SC0052125-135324A8F2D0",
  "coordinator_id": "amelia",
  "report_reference": "SC0052125",
  "revision_number": 1,
  "street_name": "Market Place",
  "district": "Town Centre",
  "assessment": "ORDER REPAIR",
  "decisions_open": ["ORDER REPAIR", "INSPECT ON SITE", "… 1 more"],
  "requirements_met_count": 4,
  "coordination_triggers": [],
  "live_permits": ["SWP0071384"],
  "repeat_likelihood": 0.03,
  "likelihood_band": "LOW",
  "crew_hours_text": "3.1 crew hours against 2.5 planned: no overrun is expected",
  "recommended_decision": "ORDER REPAIR",
  "status": "AWAITING_DECISION",
  "llm_call_count": 5,
  "decision": null,
  "decided_by": null
}
```


**How it is handled:** the review record has 45 fields; the main ones are shown. The decision fields and the documents are all `null` until an inspector decides. The thread id is the reference plus 12 hex characters, so the paused run can be resumed after a restart.

### 422 Request shorter than 5 characters

```
$ curl -s -X POST "$B/reviews" -H "X-Session-Token: $TA" -H "Content-Type: application/json" -d
'{"report_reference": "SC0052125", "coordinator_request": "ok"}'
```


```
{
  "detail": [{"type": "string_too_short", "loc": ["body", "coordinator_request"], "msg": "String should have
at least 5 characters", "input": "ok", "ctx": {"min_length": 5}}]
}
```


**How it is handled:** schema refusal: no run is opened.

### 404 Report not in the queue

```
$ curl -s -X POST "$B/reviews" -H "X-Session-Token: $TA" -H "Content-Type: application/json" -d
'{"report_reference": "SC0000000", "coordinator_request": "Are the requirements of the standard met, and
does any document disagree with the report?"}'
```

```
{
  "detail": "No open defect report has the reference SC0000000"
}
```

**How it is handled:** checked before any run starts.

### 403 Inspector outside the street's district

```
$ curl -s -X POST "$B/reviews" -H "X-Session-Token: $TP" -H "Content-Type: application/json" -d
'{"report_reference": "SC0052125", "coordinator_request": "Are the requirements of the standard met, and
does any document disagree with the report?"}'
```

```
{
  "detail": "priya is appointed for North and may not act on report SC0052125, which is on Market Place"
}
```

**How it is handled:** an inspector acts only on the streets of their own district; Market Place is Town Centre. No run is opened.

### 403 Adviser tries to start a review

```
$ curl -s -X POST "$B/reviews" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"report_reference": "SC0052125", "coordinator_request": "Are the requirements of the standard met, and
does any document disagree with the report?"}'
```

```
{
  "detail": "The Customer Services Adviser role does not include the review permission"
}
```

**How it is handled:** advisers have no **review** permission.

### 409 Second review of the same revision

```
$ curl -s -X POST "$B/reviews" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"report_reference": "SC0052125", "coordinator_request": "Are the requirements of the standard met, and
does any document disagree with the report?"}'
```

```
{
  "detail": "Revision 1 of report SC0052125 has already been reviewed; each revision is reviewed once"
}
```

**How it is handled:** under the report's lock, a revision that already has a stored review is refused before the workflow starts: no run, no call.

### 502 Gemini key not set: the model call fails

```
$ curl -s -X POST "$B/reviews" -H "X-Session-Token: $TA" -H "Content-Type: application/json" -d
'{"report_reference": "SC0052125", "coordinator_request": "Are the requirements of the standard met, and
does any document disagree with the report?"}'
```

```
{
  "detail": "400 INVALID_ARGUMENT. {'error': {'code': 400, 'message': 'API key not valid. Please pass a valid
API key.', 'status': 'INVALID_ARGUMENT', 'details': [{'@type': 'type.goog…"
}
```

**How it is handled:** the real reply with the placeholder key (detail trimmed). The failure handler answers 502 with the failure's own message and marks the run **FAILED** with **0** calls recorded (RUN-9A2F1E1C9652). No review is stored, so the report stays *awaiting review* and can be started again; the dashboard wraps it as "*The review could not be completed: … the review can be started again.*"

# GET /reviews · list reviews by status

Query: `status_value`, required, repeated for each status: `AWAITING_DECISION`, `REPAIR_ORDERED`, `INSPECTION_REQUESTED`, `REFERRED_FOR_COORDINATION`. Sorted by report reference.

### 200 Decision queue: reviews awaiting a decision

stand-in model wording

```
$ curl -s "$B/reviews?status_value=AWAITING_DECISION" -H "X-Session-Token: $TO"
```


```
[
  {"review_id": "REV-ABDB38840557", "run_id": "RUN-2BC6397CA974", "thread_id": "SC0052125-135324A8F2D0",
"coordinator_id": "amelia", "report_reference": "SC0052125", "revision_number": 1, "street_name": "Market
Place", "district": "Town Centre", "assessment": "ORDER REPAIR", "decisions_open": ["ORDER REPAIR", "… 2
more"], "requirements_met_count": 4, "coordination_triggers": [], "live_permits": ["SWP0071384"],
"repeat_likelihood": 0.03, "likelihood_band": "LOW", "crew_hours_text": "3.1 crew hours against 2.5 planned:
no overrun is expected", "recommended_decision": "ORDER REPAIR", "status": "AWAITING_DECISION",
"llm_call_count": 5, "decision": null, "decided_by": null}
]
```


**How it is handled:** what the decision queue reads (the dashboard then keeps the inspector's own streets).

### 200 Decided reviews

```
$ curl -s "$B/reviews?
status_value=REPAIR_ORDERED&status_value=INSPECTION_REQUESTED&status_value=REFERRED_FOR_COORDINATION" -H "X-
Session-Token: $TG"
```


```
[
  {"review_id": "REV-ABDB38840557", "report_reference": "SC0052125", "status": "REPAIR_ORDERED", "decision":
"ORDER REPAIR", "decided_by": "owen", "llm_call_count": 7},
  {"review_id": "REV-1F26D6510701", "report_reference": "SC0052144", "status": "REFERRED_FOR_COORDINATION",
"decision": "REFER FOR COORDINATION", "decided_by": "owen", "llm_call_count": 5},
  {"review_id": "REV-60343EE425E1", "report_reference": "SC0052186", "status": "INSPECTION_REQUESTED",
"decision": "INSPECT ON SITE", "decided_by": "owen", "llm_call_count": 6}
]
```


**How it is handled:** three reviews, one per decision, captured after the end-to-end run.

### 422 A status that does not exist

```
$ curl -s "$B/reviews?status_value=DONE" -H "X-Session-Token: $TG"
```


```
{
  "detail": [{"type": "literal_error", "loc": ["query", "status_value", "… 1 more"], "msg": "Input should be
'AWAITING_DECISION', 'REPAIR_ORDERED', 'INSPECTION_REQUESTED' or 'REFERRED_FOR_COORDINATION'", "input":
"DONE", "ctx": {"expected": "'AWAITING_DECISION', 'REPAIR_ORDERED', 'INSPECTION_REQUESTED' or
'REFERRED_FOR_COORDINATION'"}}]
}
```


**How it is handled:** only the four statuses exist.

### 422 No status_value

```
$ curl -s "$B/reviews" -H "X-Session-Token: $TG"
```


```
{
  "detail": [{"type": "missing", "loc": ["query", "status_value"], "msg": "Field required", "input": null}]
}
```


**How it is handled:** the status list is required.

### 403 Adviser lists reviews

```
$ curl -s "$B/reviews?status_value=AWAITING_DECISION" -H "X-Session-Token: $TM"
```

```
{
  "detail": "The Customer Services Adviser role does not include the audit permission"
}
```

**How it is handled:** listing reviews needs the **audit** permission.

# POST /reviews/{review_id}/decision · the inspector decides

Body: `decision` (one of `ORDER REPAIR`, `INSPECT ON SITE`, `REFER FOR COORDINATION`) and `decision_reason` (10–1000 characters). Every check below runs **before** the paused run resumes, so a refused decision never spends a model call. Then the run writes what the decision calls for and records everything in one transaction.

**200** **owen orders the repair** — stand-in model wording

```
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TO" -H "Content-Type: application/json" -d '{"decision": "ORDER REPAIR", "decision_reason": "Every requirement of the standard is met on the report and no coordination trigger applies."}'
```


```
{
  "review_id": "REV-ABDB38840557",
  "report_reference": "SC0052125",
  "status": "REPAIR_ORDERED",
  "decision": "ORDER REPAIR",
  "decided_by": "owen",
  "decided_by_role": "Highways Inspector",
  "decided_by_district": "Town Centre",
  "decision_reason": "Every requirement of the standard is met on the report and no coordination trigger applies.",
  "llm_call_count": 7,
  "order_document_text": "SUBJECT: Works for SC0052125\n\nDOCUMENT: Please carry out the works. - Report reference: SC0052125 - Defect type: Footway - Linked defect type: No linked defect - Street: Market Place - Location: oppos…",
  "document_details": {"report_reference": "SC0052125", "defect_type": "Footway", "linked_defect_type": "No linked defect", "street_name": "Market Place", "location_description": "opposite no. 50", "measurements": {"Footway": {"defect_depth_mm": 20, "defect_width_mm": 400}}},
  "resident_update_text": "HEADLINE: Repair ordered for SC0052125\n\nUPDATE: Thank you for your report. A repair has been ordered. - Report reference: SC0052125 - Defect type: Footway - Linked defect type: No linked defect - Stre…",
  "coordination_brief": null
}
```


**How it is handled:** ORDER REPAIR → works order (order writer agent) → resident update (update writer agent) → status REPAIR_ORDERED, 7 calls in all. `document_details` are built by code from the report, never by the model.

### 200 owen requests an inspection

stand-in model wording

```
$ curl -s -X POST "$B/reviews/$REV2/decision" -H "X-Session-Token: $TO" -H "Content-Type: application/json"
-d '{"decision": "INSPECT ON SITE", "decision_reason": "The depth is not documented on the report, so it
must be measured on site."}'
```


```
{
  "review_id": "REV-60343EE425E1",
  "report_reference": "SC0052186",
  "status": "INSPECTION_REQUESTED",
  "decision": "INSPECT ON SITE",
  "decided_by": "owen",
  "decided_by_role": "Highways Inspector",
  "decided_by_district": "Town Centre",
  "decision_reason": "The depth is not documented on the report, so it must be measured on site.",
  "llm_call_count": 6,
  "order_document_text": "SUBJECT: Works for SC0052186\n\nDOCUMENT: Please carry out the works. - Report
reference: SC0052186 - Standards screened: Defect-Standard-Carriageway.pd…",
  "document_details": {"report_reference": "SC0052186", "defect_types": ["Carriageway", "… 1 more"],
"requirements_to_settle": [{"defect_type": "Carriageway", "requirement_name": "Depth",
"requirement_sentence": "The defect has a depth or a vertical step of at least 40 millimetres.", "result":
"Not met", "report_answers": {"defect_depth_mm": 38}, "blank_answers": []}], "inspection_records":
{"Carriageway": "When a report is sent for inspection on site, the inspector who visits records what this
standard needs to decide the repair: the depth and the width …", "Drainage": "When a report is sent for
inspection on site, the inspector who visits records what this standard needs to decide the repair: how deep
the water stand…"}, "flagged_statements": [{"document_id": "DOC-841207", "sentence": "The caller says it is
50 mm deep in the middle.", "flag": "DISAGREES WITH FORM"}]},
  "resident_update_text": null,
  "coordination_brief": null
}
```


**How it is handled:** INSPECT ON SITE → inspection request naming every requirement to settle, the answers needed and what the inspector records → INSPECTION_REQUESTED, 6 calls.

### 200 owen refers it for coordination

```
$ curl -s -X POST "$B/reviews/$REV3/decision" -H "X-Session-Token: $TO" -H "Content-Type: application/json"
-d '{"decision": "REFER FOR COORDINATION", "decision_reason": "A coordination trigger applies, so the
network team must fit the repair around it."}'
```


```
{
  "review_id": "REV-1F26D6510701",
  "report_reference": "SC0052144",
  "status": "REFERRED_FOR_COORDINATION",
  "decision": "REFER FOR COORDINATION",
  "decided_by": "owen",
  "decided_by_role": "Highways Inspector",
  "decided_by_district": "Town Centre",
  "decision_reason": "A coordination trigger applies, so the network team must fit the repair around it.",
  "llm_call_count": 5,
  "order_document_text": null,
  "document_details": null,
  "resident_update_text": null,
  "coordination_brief": {"report_reference": "SC0052144", "defect_type": "Drainage", "linked_defect_type":
"No linked defect", "street_name": "Bridge Street", "district": "Town Centre", "street_category": "Traffic
sensitive", "location_description": "opposite no. 169", "requirements": [{"defect_type": "Drainage",
"requirement_name": "Location fixed", "result": "Met", "blank_answers": []}, "… 3 more"],
"coordination_triggers": ["traffic-sensitive street"], "live_permits": ["SWP0071351"], "conflicting_permits":
[], "flagged_statements": [{"document_id": "DOC-841256", "sentence": "There is 80 mm of standing water by the
kerb.", "flag": "DISAGREES WITH FORM"}], "repeat_likelihood": 0.77, "likelihood_band": "HIGH",
"expected_crew_hours": 6.2, "planned_crew_hours": 5.0, "overrun_expected": false}
}
```


**How it is handled:** REFER FOR COORDINATION → no document is written; a coordination brief is built by code (details, requirements, triggers, permits, flags, figures) → REFERRED_FOR_COORDINATION, 5 calls.

### 409 amelia decides her own review

```
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TA" -H "Content-Type: application/json" -d '{"decision": "ORDER REPAIR", "decision_reason": "Every requirement of the standard is met on the report and no coordination trigger applies."}'
```

```
{
  "detail": "amelia opened the review of report SC0052125 and may not decide it"
}
```

**How it is handled:** the inspector who opened a review may not decide it: the other inspector of the district must.

### 403 priya (North) decides a Town Centre review

```
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TP" -H "Content-Type: application/json" -d '{"decision": "ORDER REPAIR", "decision_reason": "Every requirement of the standard is met on the report and no coordination trigger applies."}'
```

```
{
  "detail": "priya is appointed for North and may not act on report SC0052125, which is on Market Place"
}
```

**How it is handled:** district checked before the decision is looked at.

### 403 A coordinator tries to decide

```
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d '{"decision": "ORDER REPAIR", "decision_reason": "Every requirement of the standard is met on the report and no coordination trigger applies."}'
```

```
{
  "detail": "The Street Care Coordinator role does not include the decision permission"
}
```

**How it is handled:** coordinators have no **decision** permission.

### 422 Reason shorter than 10 characters

```
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TO" -H "Content-Type: application/json" -d '{"decision": "ORDER REPAIR", "decision_reason": "Fine."}'
```

```
{
  "detail": [{"type": "string_too_short", "loc": ["body", "decision_reason"], "msg": "String should have at least 10 characters", "input": "Fine.", "ctx": {"min_length": 10}}]
}
```

**How it is handled:** schema refusal.

### 422 A decision that is none of the three

```
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TO" -H "Content-Type: application/json" -d '{"decision": "PATCH IT", "decision_reason": "Every requirement of the standard is met on the report and no coordination trigger applies."}'
```

```
{
  "detail": [{"type": "literal_error", "loc": ["body", "decision"], "msg": "Input should be 'ORDER REPAIR', 'INSPECT ON SITE' or 'REFER FOR COORDINATION'", "input": "PATCH IT", "ctx": {"expected": "'ORDER REPAIR', 'INSPECT ON SITE' or 'REFER FOR COORDINATION'"}}]
}
```

**How it is handled:** schema refusal: only the three decisions exist.

### 404 A review that does not exist

```
$ curl -s -X POST "$B/reviews/REV-DOES-NOT-EXIST/decision" -H "X-Session-Token: $TO" -H "Content-Type: application/json" -d '{"decision": "ORDER REPAIR", "decision_reason": "Every requirement of the standard is met on the report and no coordination trigger applies."}'
```

```
{
  "detail": "No report review has that identifier"
}
```

**How it is handled:** unknown review id.

### 409 owen tries ORDER REPAIR on it

```
$ curl -s -X POST "$B/reviews/$REV2/decision" -H "X-Session-Token: $TO" -H "Content-Type: application/json" -d '{"decision": "ORDER REPAIR", "decision_reason": "Every requirement of the standard is met on the report and no coordination trigger applies."}'
```

```
{
  "detail": "ORDER REPAIR is not open for report SC0052186: a requirement is Not met or Not documented"
}
```

**How it is handled:** SC0052186 was assessed INSPECT ON SITE (a requirement is not met), so only INSPECT ON SITE and REFER FOR COORDINATION are open.

### 409 Deciding the same review again

```
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TO" -H "Content-Type: application/json" -d '{"decision": "ORDER REPAIR", "decision_reason": "Every requirement of the standard is met on the report and no coordination trigger applies."}'
```

```
{
  "detail": "The review of report SC0052125 has already been decided"
}
```

**How it is handled:** a review is decided once.

## Refusals of this endpoint that need a broken state to see

| Code | Message | When |
|---|---|---|
| 409 | "*owen has sent INSPECT ON SITE for report … and it is still being recorded; only owen can finish it, by sending the same decision with the same reason again*" | A decision was accepted but writing its document failed (e.g. Gemini error). The same person sending the same decision again resumes from the checkpoint and finishes it; anyone else is refused. |
| 502 | "*Review REV-… has no workflow checkpoint in database/checkpoint.db*" | `checkpoint.db` was deleted while a review was waiting. Reset both databases (Step 3). |
| 502 | "*Review REV-… is of report …, which is not in the report queue*" | The live files changed and the report is no longer open. |
| 502 | The Gemini error, or e.g. "*order_writer_agent did not end its document with Street Care Office, Wrenford Borough Council*" | A model failure or an agent reply that breaks its rules (one-line subject naming the reference, every defect type and street named, no square brackets, no *likelihood* / *probability* / *expected crew hours*, no staff names, sign-off last). The dashboard says "*The decision could not be finished: … Record the same decision with the same reason again to finish it.*" |

# GET /reviews/{review_id}/calls · the call budget

Every model call recorded for the review's run, with tokens and latency, against the number the workflow allows: 5 before a decision, then 7 / 6 / 5 for ORDER / INSPECT / REFER.

### 200 Calls at the gate

stand-in model wording

```
$ curl -s "$B/reviews/$REV/calls" -H "X-Session-Token: $TG"
```


```
{
  "review_id": "REV-ABDB38840557",
  "run_id": "RUN-2BC6397CA974",
  "call_count": 5,
  "expected_call_count": 5,
  "total_tokens": 3486,
  "total_latency_milliseconds": 5,
  "calls": [{"call_id": 1, "run_id": "RUN-2BC6397CA974", "agent_name": "review_planner_agent", "model_name":
"fake-model", "prompt_tokens": 533, "response_tokens": 29, "total_tokens": 562, "latency_milliseconds": 1},
{"call_id": 2, "run_id": "RUN-2BC6397CA974", "agent_name": "requirements_analyst_agent", "model_name": "fake-
model", "prompt_tokens": 580, "response_tokens": 35, "total_tokens": 615, "latency_milliseconds": 1}, "… 3
more"]
}
```


**How it is handled:** at the gate: five calls, in agent order planner → requirements → evidence → risk → writer.

### 200 Calls after the order

stand-in model wording

```
$ curl -s "$B/reviews/$REV/calls" -H "X-Session-Token: $TG"
```


```
{
  "review_id": "REV-ABDB38840557",
  "run_id": "RUN-2BC6397CA974",
  "call_count": 7,
  "expected_call_count": 7,
  "total_tokens": 4218,
  "total_latency_milliseconds": 7,
  "calls": ["review_planner_agent", "requirements_analyst_agent", "evidence_analyst_agent",
"risk_analyst_agent", "review_writer_agent", "order_writer_agent", "resident_update_writer_agent"]
}
```


**How it is handled:** after ORDER REPAIR: the order writer and update writer are added; totals are the sums of the calls.

### 200 Calls after an inspection

```
$ curl -s "$B/reviews/$REV2/calls" -H "X-Session-Token: $TG"
```


```
{
  "review_id": "REV-60343EE425E1",
  "run_id": "RUN-1D9C2681EE10",
  "call_count": 6,
  "expected_call_count": 6,
  "total_tokens": 4644,
  "total_latency_milliseconds": 6
}
```


**How it is handled:** after INSPECT ON SITE: 6 of 6.

### 200 Calls after a referral

```
$ curl -s "$B/reviews/$REV3/calls" -H "X-Session-Token: $TG"
```


```
{
  "review_id": "REV-1F26D6510701",
  "run_id": "RUN-0810FE291999",
  "call_count": 5,
  "expected_call_count": 5,
  "total_tokens": 3561,
  "total_latency_milliseconds": 5
}
```


**How it is handled:** after REFER FOR COORDINATION: 5 of 5.

### 404 Calls of an unknown review

```
$ curl -s "$B/reviews/REV-DOES-NOT-EXIST/calls" -H "X-Session-Token: $TG"
```


```
{
  "detail": "No report review has that identifier"
}
```


**How it is handled:** unknown review id.

### 403 Adviser reads calls

```
$ curl -s "$B/reviews/$REV/calls" -H "X-Session-Token: $TM"
```


```
{
  "detail": "The Customer Services Adviser role does not include the audit permission"
}
```


**How it is handled:** needs the **audit** permission.

# POST /chat/ask · the street care assistant

Body: `question` (3–1000 characters). One model call reads the question with the 2 searches' passages and the asker's last 3 exchanges and classifies it into one of 7 types. Questions about the documents are answered by the model *only* when the nearest passage is within 0.64; questions about one report or a street are then answered **by code from the records**, within what the role allows; everything else is refused. Every reply is recorded as a chat turn.

### 200 A defect standard question

stand-in model wording

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"question": "What does the footway defect standard ask for before a repair is ordered?"}'
```


```
{
  "question_type": "DEFECT STANDARD",
  "report_reference": null,
  "street_name": null,
  "defect_type": "Footway",
  "answer": "Under the footway standard the location must be fixed to a house number, a lamp column or a
junction, a photograph is attached, and the depth or vertical step and the width are measured.",
  "source_documents": ["Defect-Standard-Footway.pdf", "Defect-Standard-Carriageway.pdf", "… 1 more"],
  "was_refused": false
}
```


**How it is handled:** answered from the documents: `source_documents` names the standard and the documents of the passages found (never the casebook).

### 200 A street care rules question

stand-in model wording

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"question": "Who decides a review?"}'
```


```
{
  "question_type": "STREET CARE RULES",
  "report_reference": null,
  "street_name": null,
  "defect_type": null,
  "answer": "A Highways Inspector appointed for the street's district, who did not open the review, decides
it.",
  "source_documents": ["Defect-Standard-Footway.pdf", "Defect-Standard-Carriageway.pdf", "… 2 more"],
  "was_refused": false
}
```


**How it is handled:** STREET CARE RULES: answered from the Highways Policy passages.

### 200 A statement guide question

stand-in model wording

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"question": "How does the guide say a measurement statement is checked?"}'
```


```
{
  "question_type": "STATEMENT GUIDANCE",
  "report_reference": null,
  "street_name": null,
  "defect_type": null,
  "answer": "A measurement sentence is checked against the figures on the report, and a disagreement is left
to the Highways Inspector.",
  "source_documents": ["Guide-Measurement.pdf", "Guide-Location.pdf", "Guide-Photograph.pdf", "… 7 more"],
  "was_refused": false
}
```


**How it is handled:** STATEMENT GUIDANCE: every statement guide is cited.

### 200 Out of scope

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"question": "What will the weather be like tomorrow?"}'
```


```
{
  "question_type": "OUT OF SCOPE",
  "report_reference": null,
  "street_name": null,
  "defect_type": null,
  "answer": "The street care documents do not answer that question.",
  "source_documents": [],
  "was_refused": true
}
```


**How it is handled:** not about the documents → OUT OF SCOPE, `was_refused` true, the refusal sentence.

### 200 Adviser asks the chat while the review waits

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question": "What is the status of report SC0052125?"}'
```


```
{
  "question_type": "REPORT STATUS",
  "report_reference": "SC0052125",
  "street_name": null,
  "defect_type": null,
  "answer": "SC0052125 (Footway) on Market Place is awaiting a decision.",
  "source_documents": [],
  "was_refused": false
}
```


**How it is handled:** a Customer Services Adviser hears only the defect types, street and status.

### 200 Coordinator asks the chat while the review waits

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"question": "What is the status of SC-52125?"}'
```


```
{
  "question_type": "REPORT STATUS",
  "report_reference": "SC0052125",
  "street_name": null,
  "defect_type": null,
  "answer": "REV-ABDB38840557, report SC0052125 revision 1 (Footway) on Market Place, opened by amelia:
awaiting a decision.\nAssessment ORDER REPAIR. Footway: Location fixed Met, Photograph Met, Depth Met, Width
Met. Coordination triggers: non…",
  "source_documents": [],
  "was_refused": false
}
```


**How it is handled:** an inspector or coordinator hears the six-line review record. `SC-52125` was read as SC0052125.

### 200 Adviser asks for a prediction

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question": "Will report SC0052125 come back?"}'
```


```
{
  "question_type": "OUTCOME PREDICTION",
  "report_reference": "SC0052125",
  "street_name": null,
  "defect_type": null,
  "answer": "A defect report is decided by a Highways Inspector, and customer services is not told whether a
defect is likely to come back or how long its repair will take.",
  "source_documents": [],
  "was_refused": false
}
```


**How it is handled:** advisers are never told a prediction, before anything else is looked at.

### 200 Coordinator asks for a prediction

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"question": "Will report SC0052158 come back, and how long will it take?"}'
```


```
{
  "question_type": "OUTCOME PREDICTION",
  "report_reference": "SC0052158",
  "street_name": null,
  "defect_type": null,
  "answer": "No review of report SC0052158 has been recorded: it is awaiting review.",
  "source_documents": [],
  "was_refused": false
}
```


**How it is handled:** the team hears the record; this report has no review yet.

### 200 A report that is not open

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"question": "What is the status of SC0099999?"}'
```


```
{
  "question_type": "REPORT STATUS",
  "report_reference": "SC0099999",
  "street_name": null,
  "defect_type": null,
  "answer": "No open defect report is recorded under reference SC0099999.",
  "source_documents": [],
  "was_refused": false
}
```


**How it is handled:** not an open report.

### 200 A status question naming no report

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question": "What is the status of my report?"}'
```


```
{
  "question_type": "REPORT STATUS",
  "report_reference": null,
  "street_name": null,
  "defect_type": null,
  "answer": "Name the defect report by its reference, as SC0052186, SC-52186 or 52186.",
  "source_documents": [],
  "was_refused": false
}
```


**How it is handled:** a status question that names no report.

### 200 Street figure, enough reports

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question": "How many footway defects have been reported on High Street?"}'
```


```
{
  "question_type": "STREET FIGURE",
  "report_reference": null,
  "street_name": "High Street",
  "defect_type": "Footway",
  "answer": "801 footway defects have been reported on High Street.",
  "source_documents": [],
  "was_refused": false
}
```


**How it is handled:** counted by code from the decided report history: distinct references on that street with that defect type (linked-only reports don't count).

### 200 Street figure, fewer than five reports

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question": "How many footway defects have been reported on Kiln Lane?"}'
```


```
{
  "question_type": "STREET FIGURE",
  "report_reference": null,
  "street_name": "Kiln Lane",
  "defect_type": "Footway",
  "answer": "Figures about defects on a street are shared only where at least five reports make them up.",
  "source_documents": [],
  "was_refused": false
}
```


**How it is handled:** Kiln Lane has 2 footway reports: below 5 the figure could point to a household, so everyone hears the group-size line.

### 200 Street figure naming no street

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question": "How many footway defects have been reported?"}'
```


```
{
  "question_type": "STREET FIGURE",
  "report_reference": null,
  "street_name": null,
  "defect_type": "Footway",
  "answer": "Name one street, as High Street, and one defect type: carriageway, footway, drainage, vegetation
or street lighting.",
  "source_documents": [],
  "was_refused": false
}
```


**How it is handled:** a street figure needs one street and one defect type.

### 422 Question shorter than 3 characters

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question": "ok"}'
```


```
{
  "detail": [{"type": "string_too_short", "loc": ["body", "question"], "msg": "String should have at least 3
characters", "input": "ok", "ctx": {"min_length": 3}}]
}
```


**How it is handled:** schema refusal: nothing reaches the model and nothing is recorded.

### 422 Question longer than 1000 characters

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d "
{\"question\":\"$(printf 'a%.0s' {1..1001})\"}"
```


```
{
  "detail": [{"type": "string_too_long", "loc": ["body", "question"], "msg": "String should have at most 1000
characters", "input":
"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaa…", "ctx": {"max_length": 1000}}]
}
```


**How it is handled:** schema refusal (the dashboard cannot even type it).

### 502 Gemini key not set: the chat call fails

```
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question": "What does the footway standard ask for?"}'
```


```
{
  "detail": "400 INVALID_ARGUMENT. {'error': {'code': 400, 'message': 'API key not valid. Please pass a valid
API key.', 'status': 'INVALID_ARGUMENT', 'details': [{'@type': 'type.goog…"
}
```


**How it is handled:** the real reply with the placeholder key: a failed request records no chat turn. The dashboard shows "*The question could not be answered: … Ask it again.*"

# GET /chat/turns · my own conversation

Query: `turn_limit`, required, 1–200. Only the signed-in person's turns, newest first. `source_documents` is one comma-separated string here.

### 200 Own turns

```
$ curl -s "$B/chat/turns?turn_limit=2" -H "X-Session-Token: $TM"
```


```
[
  {"turn_id": 15, "username": "maya", "user_role": "Customer Services Adviser", "question_text": "How many
footway defects have been reported?", "question_type": "STREET FIGURE", "report_reference": null,
"street_name": null, "defect_type": "Footway", "answer_text": "Name one street, as High Street, and one
defect type: carriageway, footway, drainage, vegetation or street lighting.", "source_documents": "",
"closest_distance": 0.33864152431488037, "was_refused": false},
  {"turn_id": 14, "username": "maya", "user_role": "Customer Services Adviser", "question_text": "How many
footway defects have been reported on Kiln Lane?", "question_type": "STREET FIGURE", "report_reference":
null, "street_name": "Kiln Lane", "defect_type": "Footway", "answer_text": "Figures about defects on a street
are shared only where at least five reports make them up.", "source_documents": "", "closest_distance":
0.32289761304855347, "was_refused": false}
]
```


**How it is handled:** maya's last 2 turns; nobody else's.

### 422 turn_limit 0

```
$ curl -s "$B/chat/turns?turn_limit=0" -H "X-Session-Token: $TM"
```


```
{
  "detail": [{"type": "greater_than_equal", "loc": ["query", "turn_limit"], "msg": "Input should be greater
than or equal to 1", "input": "0", "ctx": {"ge": 1}}]
}
```


**How it is handled:** below 1.

### 422 turn_limit 201

```
$ curl -s "$B/chat/turns?turn_limit=201" -H "X-Session-Token: $TM"
```


```
{
  "detail": [{"type": "less_than_equal", "loc": ["query", "turn_limit"], "msg": "Input should be less than or
equal to 200", "input": "201", "ctx": {"le": 200}}]
}
```


**How it is handled:** above 200.

### 422 No turn_limit

```
$ curl -s "$B/chat/turns" -H "X-Session-Token: $TM"
```


```
{
  "detail": [{"type": "missing", "loc": ["query", "turn_limit"], "msg": "Field required", "input": null}]
}
```


**How it is handled:** required.

# GET /audit/chat-turns · everyone's latest questions

Query: `turn_limit` (1–200). Turns from every person, newest first, for the audit trail.

### 200 Everyone's latest turns

```
$ curl -s "$B/audit/chat-turns?turn_limit=2" -H "X-Session-Token: $TG"
```


```
[
  {"turn_id": 15, "username": "maya", "user_role": "Customer Services Adviser", "question_text": "How many
footway defects have been reported?", "question_type": "STREET FIGURE", "report_reference": null,
"street_name": null, "defect_type": "Footway", "answer_text": "Name one street, as High Street, and one
defect type: carriageway, footway, drainage, vegetation or street lighting.", "source_documents": "",
"closest_distance": 0.33864152431488037, "was_refused": false},
  {"turn_id": 14, "username": "maya", "user_role": "Customer Services Adviser", "question_text": "How many
footway defects have been reported on Kiln Lane?", "question_type": "STREET FIGURE", "report_reference":
null, "street_name": "Kiln Lane", "defect_type": "Footway", "answer_text": "Figures about defects on a street
are shared only where at least five reports make them up.", "source_documents": "", "closest_distance":
0.32289761304855347, "was_refused": false}
]
```


**How it is handled:** includes grace's and maya's turns alike.

### 403 Adviser reads the audit trail

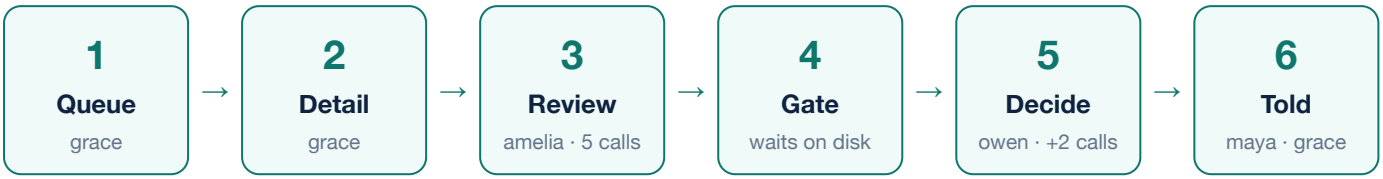
```
$ curl -s "$B/audit/chat-turns?turn_limit=2" -H "X-Session-Token: $TM"
```


```
{
  "detail": "The Customer Services Adviser role does not include the audit permission"
}
```


**How it is handled:** needs the **audit** permission.

# End-to-end: one report from the queue to a works order

Report **SC0052125**: a footway defect on Market Place (Town Centre), assessed **ORDER REPAIR**. Amelia opens the review, Owen (the other Town Centre inspector) orders the repair, and a caller is told what the resident was told. Every step below is the captured request and reply; run them in order after the setup block of Step 5.




## 1 · The coordinator finds it in the queue

```
$ curl -s "$B/queue?defect_type=Footway" -H "X-Session-Token: $TG"
```

✓ ITS ROW BEFORE ANY REVIEW (SHAPE OF EVERY QUEUE ROW)


```
{
  "queue_position": 3,
  "report_reference": "SC0052125",
  "revision_number": 1,
  "defect_type": "Footway",
  "linked_defect_type": "No linked defect",
  "street_name": "Market Place",
  "district": "Town Centre",
  "report_channel": "StreetCare app",
  "report_status": "AWAITING_REVIEW",
  "document_count": 2,
  "requirements_met_count": 4,
  "requirements_not_met_count": 0,
  "requirements_not_documented_count": 0,
  "live_permit_count": 1,
  "conflicting_permit_count": 0,
  "coordination_trigger_count": 0,
  "assessment": "ORDER REPAIR",
  "repeat_likelihood": 0.03,
  "likelihood_band": "LOW",
  "expected_crew_hours": 3.1,
  "overrun_expected": false,
  "disagreement_count": 0
}
```


## 2 · The detail shows what the review will work from

```
$ curl -s "$B/queue/SC0052125" -H "X-Session-Token: $TG"
```

✓ KEY FIGURES (FROM THE TOOLS, NO MODEL)


```
{"report_status": "AWAITING_REVIEW", "assessment": "ORDER REPAIR", "requirements": [["Location fixed",
"Met"], ["Photograph", "Met"], ["Depth", "Met"], ["Width", "Met"]], "live_permits": ["SWP0071384"],
"likelihood": {"repeat_likelihood": 0.03, "likelihood_band": "LOW"}, "crew_hours": {"expected_crew_hours":
3.1, "planned_crew_hours": 2.5, "overrun_expected": false}}
```


## 3 · Two people who may not open it, then Amelia who may

```
$ curl -s -X POST "$B/reviews" -H "X-Session-Token: $TP" -H "Content-Type: application/json" -d
'{"report_reference":"SC0052125","coordinator_request":"Are the requirements of the standard met, and
does any document disagree with the report?"}'
$ REV=$(curl -s -X POST "$B/reviews" -H "X-Session-Token: $TA" -H "Content-Type: application/json" -d
'{"report_reference":"SC0052125","coordinator_request":"Are the requirements of the standard met, and
does any document disagree with the report?"}' | python3 -c "import
sys,json;print(json.load(sys.stdin)['review_id'])"); echo $REV
```

**✓ PRIYA (NORTH) → 403 · AMELIA → 201**

```
{"detail": "priya is appointed for North and may not act on report SC0052125, which is on Market Place"}
REV-ABDB38840557
```

## 4 · At the gate: 5 calls, nothing decided, everyone hears the right amount

```
$ curl -s "$B/reviews/$REV/calls" -H "X-Session-Token: $TG"
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question":"What is the status of report SC0052125?"}'
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"question":"What is the status of SC-52125?"}'
```

**✓ CALLS · ADVISER · COORDINATOR**

```
call_count 5 of expected 5: review_planner_agent, requirements_analyst_agent, evidence_analyst_agent,
risk_analyst_agent, review_writer_agent

SC0052125 (Footway) on Market Place is awaiting a decision.

REV-ABDB38840557, report SC0052125 revision 1 (Footway) on Market Place, opened by amelia: awaiting a
decision.
Assessment ORDER REPAIR. Footway: Location fixed Met, Photograph Met, Depth Met, Width Met. Coordination
triggers: none. The decisions open: ORDER REPAIR, INSPECT ON SITE, REFER FOR COORDINATION.
Street works permits live on the street: SWP0071384.
Repeat likelihood 0.03 (LOW). Expected 3.1 crew hours against 2.5 planned: no overrun is expected.
Statement evidence: 9 sentences read (Hazard, Traffic and use, Background, Measurement, Photograph);
statement sentences flagged against the report: 0.
It waits for a Highways Inspector's decision; the review recommends ORDER REPAIR.
```

## 5 · Only the right inspector can decide

```
$ D='{"decision":"ORDER REPAIR","decision_reason":"Every requirement of the standard is met on the
report and no coordination trigger applies."}'
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TA" -H "Content-Type:
application/json" -d "$D"
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TP" -H "Content-Type:
application/json" -d "$D"
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TO" -H "Content-Type:
application/json" -d "$D"
```

**✓ AMELIA → 409 · PRIYA → 403 · OWEN → 200**

```
{"detail": "amelia opened the review of report SC0052125 and may not decide it"}
{"detail": "priya is appointed for North and may not act on report SC0052125, which is on Market Place"}
{"status": "REPAIR_ORDERED", "decision": "ORDER REPAIR", "decided_by": "owen", "decided_by_district": "Town
Centre", "llm_call_count": 7}
```

## 6 · After the order: 7 calls, the record, and what each person is told

```
$ curl -s "$B/reviews/$REV/calls" -H "X-Session-Token: $TG"
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TM" -H "Content-Type: application/json" -d
'{"question":"What is the status of report SC0052125?"}'
$ curl -s -X POST "$B/chat/ask" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"question":"What is the status of SC0052125?"}'
$ curl -s -X POST "$B/reviews/$REV/decision" -H "X-Session-Token: $TO" -H "Content-Type:
application/json" -d "$D"
$ curl -s -X POST "$B/reviews" -H "X-Session-Token: $TG" -H "Content-Type: application/json" -d
'{"report_reference":"SC0052125","coordinator_request":"Look again please."}'
```

**✓ CALLS**

```
call_count 7 of expected 7: review_planner_agent, requirements_analyst_agent, evidence_analyst_agent,
risk_analyst_agent, review_writer_agent, order_writer_agent, resident_update_writer_agent
```

**✓ THE ADVISER HEARS WHERE THE REPAIR IS AND WHAT THE RESIDENT WAS TOLD (UPDATE TEXT: STAND-IN WORDING)**

```
SC0052125 (Footway) on Market Place is repair ordered. Where on the street: opposite no. 50. What the
resident has been told: Thank you for your report. A repair has been ordered. – Report reference: SC0052125
– Defect type: Footway – Linked defect type: No linked defect – Street: Market Place – Location: opposite
no. 50 Street Care Office, Wrenford Borough Council
```

**✓ THE COORDINATOR HEARS THE WHOLE RECORD, ENDING WITH THE DECISION**

```
REV-ABDB38840557, report SC0052125 revision 1 (Footway) on Market Place, opened by amelia: repair ordered.
Assessment ORDER REPAIR. Footway: Location fixed Met, Photograph Met, Depth Met, Width Met. Coordination
triggers: none. The decisions open: ORDER REPAIR, INSPECT ON SITE, REFER FOR COORDINATION.
Street works permits live on the street: SWP0071384.
Repeat likelihood 0.03 (LOW). Expected 3.1 crew hours against 2.5 planned: no overrun is expected.
Statement evidence: 9 sentences read (Hazard, Traffic and use, Background, Measurement, Photograph);
statement sentences flagged against the report: 0.
owen, Highways Inspector for Town Centre, decided ORDER REPAIR. The reason given: "Every requirement of the
standard is met on the report and no coordination trigger applies."
```

**✓ DECIDING AGAIN → 409 · REVIEWING AGAIN → 409**

```
{"detail": "The review of report SC0052125 has already been decided"}
{"detail": "Revision 1 of report SC0052125 has already been reviewed; each revision is reviewed once"}
```

Back in the queue, the row of SC0052125 now reads `"report_status": "REPAIR_ORDERED"`; nothing else in the row changed, because the queue, the detail and the stored review all come from the same tools.

## The other two decision paths

| Decision | Example | Writes | Status | Calls |
|---|---|---|---|---|
| **ORDER REPAIR** | SC0052125 (above) | works order → resident update | REPAIR_ORDERED | 7 |
| **INSPECT ON SITE** | SC0052186: carriageway + drainage, a requirement not met, 1 flagged sentence | inspection request | INSPECTION_REQUESTED | 6 |
| **REFER FOR COORDINATION** | SC0052144: drainage with a coordination trigger | nothing; coordination brief built by code | REFERRED_FOR_COORDINATION | 5 |

# Checklist: prove every safety rule works

Each row is a captured request. Every refusal happens before any model call is spent, except the one marked 502.

| Try this | Code | Reply |
|---|---|---|
| sign in with a wrong password | 401 | Username or password is not correct |
| use a token after the same account signed in again | 401 | Sign in first |
| any guarded call with no token | 401 | Sign in first |
| GET /queue as maya (adviser) | 403 | The Customer Services Adviser role does not include the queue permission |
| GET /queue/SC0000000 | 404 | No open defect report has the reference SC0000000 |
| GET /queue?defect_type=Graffiti | 422 | Input should be 'All', 'Carriageway', 'Footway', 'Drainage', 'Vegetation' or 'Street lighting' |
| POST /reviews with request "ok" | 422 | String should have at least 5 characters |
| priya (North) opens a Town Centre report | 403 | priya is appointed for North and may not act on report SC0052125, which is on Market Place |
| open a second review of the same revision | 409 | Revision 1 of report SC0052125 has already been reviewed; each revision is reviewed once |
| POST /reviews without a Gemini key | 502 | 400 INVALID_ARGUMENT. {'error': {'code': 400, 'message': 'API key not valid. Please pass a valid API key.', 'status': 'I… |
| amelia decides the review she opened | 409 | amelia opened the review of report SC0052125 and may not decide it |
| grace (coordinator) decides | 403 | The Street Care Coordinator role does not include the decision permission |
| ORDER REPAIR on a report assessed INSPECT ON SITE | 409 | ORDER REPAIR is not open for report SC0052186: a requirement is Not met or Not documented |
| decide a decided review | 409 | The review of report SC0052125 has already been decided |
| decision reason "Fine." | 422 | String should have at least 10 characters |
| GET /reviews/REV-DOES-NOT-EXIST/calls | 404 | No report review has that identifier |
| maya asks whether a report will come back | 200 | A defect report is decided by a Highways Inspector, and customer services is not told whether a defect is likely to come… |
| street figure with fewer than 5 reports | 200 | Figures about defects on a street are shared only where at least five reports make them up. |
| ask about the weather | 200 | The street care documents do not answer that question. |
| GET /chat/turns?turn_limit=201 | 422 | Input should be less than or equal to 200 |

# Troubleshooting

| You see | Fix |
| --- | --- |
| `No module named 'config'` / file not found | You are not in the `Project` folder. `cd` into it and run again. |
| `No module named 'fastapi'` (or sklearn, chromadb…) | The venv isn't active. Activate it, then `pip install -r requirements.txt`. |
| *The sentence_transformers python package is not installed* | `pip install sentence-transformers` (Step 1c). |
| Indexing step mentions `huggingface.co` | First run downloads the embedding model. Check internet / proxy and run again; afterwards it is cached. |
| *The StreetCare AI service cannot start. Missing: …* | Run the build step named next to each missing path (Step 2). |
| 502 *API key not valid* | Put a real key and model in `Project/.env` (no quotes) and restart the service. |
| 502 *… must both be set in .env* | `GEMINI_API_KEY` or `GEMINI_MODEL` is empty. |
| 502 naming an agent (e.g. *review_writer_agent recommended …*) | The model broke one of the reply rules. Nothing was stored: start the review again (or send the same decision again). |
| Tests: *No report … ORDER REPAIR is left unreviewed* | Reset the database (Step 3, red box). |
| 401 *Sign in first* with a token that worked | The account signed in again elsewhere (e.g. the dashboard). Sign in again. |
| `address already in use` | Something is on 8081 / 8080. Stop it ( `lsof -i :8081` on Mac) or close the old terminal. |
| Dashboard: *The service is not answering at http:///localhost:8081* | Start `python run_api_service.py` first, in another terminal. |
| Accounts look wrong after editing `config.py` | Setup never overwrites existing accounts: reset the database (Step 3). |

# Cheat sheet

### Once

```
$ cd Project
$ python3 -m venv venv && source
venv/bin/activate
# Windows: python -m venv venv;
venv\Scripts\Activate.ps1
$ pip install -r requirements.txt
$ pip install sentence-transformers
# fill GEMINI_API_KEY + GEMINI_MODEL in .env
$ python run_database_setup.py
$ python run_data_pipeline.py
$ python run_model_training.py
$ python run_knowledge_indexing.py
```

### Every time

```
$ cd Project
$ source venv/bin/activate
$ python -m pytest tests.py -q
$ python run_api_service.py        # terminal
1
$ streamlit run run_dashboard.py   # terminal
2
# API docs   http://localhost:8081/docs
# dashboard  http://localhost:8080
# stop with Ctrl + C
```

| It worked if you see | Value |
|---|---|
| Database setup | 6 tables · accounts created: 16 |
| Data pipeline | 30569 · 7648 · 25537 · 6337 · 35717 rows kept |
| Models | repeat AUC 0.8351 · crew hours R2 0.9025 · sentence macro F1 0.9775 |
| Knowledge index | 16 documents, 136 sections, 205 passages |
| Tests | 50 passed (40 passed + 10 failed without a key) |
| Service | Uvicorn running on http://0.0.0.0:8081 · /docs opens |
| Queue | 40 reports, footway first, positions 1–40 |
| One review | 201 · 5 calls · AWAITING_DECISION |
| Decision calls | ORDER 7 · INSPECT 6 · REFER 5 |
| Street figure | High Street footway: 801 · Kiln Lane footway: group-size line |